PW-ADR-015 · DOCUMENTO DE INGENIERIA

PitWall — Architecture Decision Records

Contexto, alternativas, decisión, razones y consecuencias

VERSION DEL DOCUMENTO
1.1

ESTADO
Vigente

FECHA
2026-09-11

AUTOR
Luis Miguel

Documento generado automaticamente a partir de `15-ADR.md`. El fichero Markdown es la unica fuente de verdad; este PDF es un artefacto derivado y no debe editarse por separado.

PitWall es un proyecto no oficial sin afiliacion con Electronic Arts Inc., Codemasters, la FIA ni Formula One World Championship Limited. Las marcas citadas pertenecen a sus respectivos titulares y se emplean unicamente de forma descriptiva.

Indice

Índice

ADR-001 — Lenguaje del núcleo: C++20/23

- Contexto
- Alternativas
- Decisión
- Razones
- Consecuencias

ADR-002 — Aplicación de escritorio Windows con WebView2

- Contexto
- Alternativas
- Decisión
- Cómo se comunican núcleo e interfaz
- Razones
- Consecuencias
- Lo que este ADR no cambia

ADR-003 — Pipeline con hilos dedicados y colas acotadas

- Contexto
- Alternativas
- Decisión
- Razones
- Consecuencias

ADR-004 — Decoder propio

- Contexto
- Alternativas
- Decisión
- Razones
- Consecuencias

ADR-005 — Almacenamiento en cuatro capas

- Contexto
- Alternativas
- Decisión
- Razones
- Consecuencias

ADR-006 — Transporte interno con canales de garantías distintas

- Contexto
- Alternativas
- Decisión
- Razones
- Consecuencias

ADR-007 — Replay como requisito del MVP

- Contexto
- Alternativas
- Decisión

Razones
Consecuencias
ADR-008 — Registro de decoders por m_packetFormat
Contexto
Alternativas
Decisión
Razones
Consecuencias
ADR-009 — Channel<T>: procedencia obligatoria en el tipo
Contexto
Alternativas
Decisión
Razones
Consecuencias
ADR-010 — El grabador va antes del decoder
Contexto
Decisión
Razones
Consecuencias
ADR-011 — Formato de captura propio en lugar de pcap
Contexto
Alternativas
Decisión
Razones
Consecuencias
ADR-012 — Sin soporte del formato 2024
Contexto
Decisión
Razones
Consecuencias
ADR-013 — Gráficas sobre canvas, no SVG
Contexto
Decisión
Razones
Consecuencias
ADR-014 — Mapa reconstruido desde telemetría
Contexto
Alternativas
Decisión
Razones
Consecuencias
ADR-015 — Fuente de verdad: el PDF de mayor versión
Contexto
Alternativas
Decisión
Razones

Consecuencias

ADR-016 — Docker: no para ejecutar ni distribuir; opcional en integración continua

Contexto

Alternativas

Decisión

Razones

Consecuencias

Nota sobre el vocabulario

Plantilla para ADR futuros

Architecture Decision Records

Índice

ADR	Decisión	Estado
ADR-001	Lenguaje del núcleo: **C++20/23**	Aceptada
ADR-002	**Aplicación de escritorio Windows con WebView2** + React/TypeScript	**Aceptada (v2)**
ADR-003	Arquitectura: **pipeline con hilos dedicados y colas acotadas**	Aceptada
ADR-004	**Decoder propio** en lugar de librería de terceros	Aceptada
ADR-005	Almacenamiento: **cuatro capas** con tecnologías distintas	Aceptada
ADR-006	Transporte **interno** hacia la interfaz, con canales de garantías distintas	Aceptada (revisado por ADR-002 v2)
ADR-007	**Replay como requisito del MVP**	Aceptada
ADR-008	Estrategia de decoders: **registro por** `m_packetFormat`	Aceptada
ADR-009	`Channel<T>`: procedencia obligatoria en el tipo	Aceptada
ADR-010	El **grabador va antes del decoder**	Aceptada
ADR-011	Formato de captura **propio** en lugar de pcap	Aceptada
ADR-012	**Sin soporte del formato 2024**	Aceptada
ADR-013	Gráficas sobre **canvas**, no SVG	Aceptada
ADR-014	Mapa **reconstruido desde telemetría**, sin imágenes	Aceptada
ADR-015	**Fuente de verdad**: el PDF de mayor versión	Aceptada
ADR-016	**Docker: no para ejecutar ni distribuir; opcional en integración continua**	**Aceptada**

ADR-001 — Lenguaje del núcleo: C++20/23

Estado: Aceptada · 2026-09-11

Contexto

El núcleo debe: decodificar structs `packed` de tamaño fijo en little-endian, sostener ~415 datagramas/s sin pérdida, coordinar seis hilos con contrapresión explícita, y ejecutarse en un PC Windows que simultáneamente está corriendo el juego. Existe además un objetivo formativo declarado: aprender C++ moderno, concurrencia, redes, optimización y sistemas de baja latencia.

Alternativas

Opción	A favor	En contra
C++20/23	Transcripción literal de la especificación con verificación en compilación; control total de hilos y memoria; sin recolector de basura; cumple el objetivo formativo	Mayor riesgo de errores de memoria; mayor pendiente de aprendizaje; ecosistema del dominio escaso
Rust	Seguridad de memoria garantizada; rendimiento equivalente	Curva alta; ecosistema del dominio también escaso; no cumple el objetivo formativo declarado
C# / .NET	El mejor ecosistema del dominio (la única librería con soporte 2026); excelente en Windows; `MemoryMarshal` da un mapeo casi 1:1	Requiere runtime; no cumple el objetivo formativo
TypeScript / Node	Un solo lenguaje con el frontend; desarrollo muy rápido	Sin control de layout binario; sin verificación de tamaños en compilación; parseo por offsets a mano
Python	El más rápido de prototipar	El GIL y el coste por paquete dejan margen estrecho a 60 Hz

Decisión

C++20/23 para el núcleo, con MSVC como compilador principal y Clang como segunda opinión.

Razones

1. **El problema encaja con el lenguaje.** Structs `packed` de tamaño fijo: `#pragma pack` más `static_assert(sizeof(T) == N)` convierte la especificación en código verificado por el compilador. Es el caso de uso canónico.
2. **El `static_assert` es, literalmente, la prueba de protocolo exigida.** Coste cero, imposible de olvidar, se ejecuta en cada compilación.
3. **Control explícito de la concurrencia**, que es un requisito duro (RNF-050) y no un detalle.
4. **El objetivo formativo es un requisito del proyecto**, no una preferencia. Un stack que lo ignore fallaría en un objetivo real.
5. **No existe librería de C++ con soporte 2026**, así que la ventaja de ecosistema de C# no es aprovechable sin cambiar de lenguaje entero.

Consecuencias

Positivas. Verificación estática del layout · latencia predecible sin recolector · un único binario nativo · control total de la memoria.

Negativas, y asumidas.

- Mayor riesgo de errores de memoria (RSK-06). **Mitigación obligatoria:** `std::span` en lugar de punteros, validación estricta de tamaño antes de toda lectura, fuzzing continuo, desinfectantes en integración continua.
- Mayor tiempo de desarrollo (RSK-04). **Mitigación:** alcance del MVP acotado por criterios objetivos y plan de fases pedagógico.
- Dos lenguajes en el proyecto. **Mitigación:** los tipos TypeScript se **generan** desde el dominio para que no puedan divergir.
- Un `reinterpret_cast` sobre buffer es técnicamente comportamiento indefinido. **Mitigación:** variante con `memcpy` activable por opción de compilación, y las pruebas se ejecutan con ambas.

ADR-002 — Aplicación de escritorio Windows con WebView2

Estado: Aceptada (v2) · 2026-09-11 **Sustituye a:** ADR-002 v1, que decidía «React en el navegador servido por el núcleo».

Contexto

La v1 de este ADR eligió servir la interfaz por HTTP en `127.0.0.1` y consumirla en el navegador. Esa decisión resolvía bien la construcción de la interfaz, pero definía mal **el producto**: obligaba al usuario a abrir un navegador, escribir o marcar una URL, y convivía con la idea de un acceso desde la LAN que nunca fue un objetivo real.

La experiencia objetivo, fijada explícitamente, es otra:

1. El usuario instala PitWall.
2. Abre `PitWall.exe`.
3. Se abre una ventana propia.
4. El núcleo ya está escuchando UDP.
5. Al abrir el juego, PitWall detecta la recepción por sí solo.
6. Detecta formato, sesión, jugador y estado sin intervención.
7. **No existe un paso manual de conexión.**
8. Si UDP no está configurado, la propia aplicación lo explica.
9. El usuario no necesita navegador, terminal, Docker, ni saber qué es un puerto.

Ninguna de esas nueve frases menciona la web. Un producto cuya interfaz vive en el navegador las contradice ya en la primera.

Alternativas

Opción	A favor	En contra
Ventana nativa + WebView2 + React	Aplicación de escritorio real, un solo proceso, un solo ejecutable; el motor viene con Windows 11; conserva íntegra la inversión en React/TypeScript; peso mínimo	Dependencia del WebView2 Runtime en Windows 10; automatización de pruebas menos trillada que en un navegador
Electron	Motor empaquetado, comportamiento idéntico en cualquier máquina, herramientas de prueba maduras	**~150 MB por aplicación**, y un proceso Node adicional que aquí no pinta nada: el núcleo ya es C++, así que Electron obligaría a un tercer runtime. Descartado salvo razón técnica muy fuerte
Tauri	Ligero y moderno	Su anfitrión es **Rust**. Con un núcleo en C++ implicaría un tercer lenguaje y un puente entre ambos
Interfaz nativa (Qt, Dear ImGui, WinUI)	Sin motor web, máximo control	Descarta la interfaz React ya diseñada; las gráficas de alta frecuencia y el mapa costarían mucho más por hora invertida, con peor resultado visual
Seguir en el navegador (v1)	Cero trabajo adicional	Contradice los nueve puntos de la experiencia objetivo

Decisión

PitWall es una aplicación de escritorio para Windows. Un único `PitWall.exe` que:

- abre una **ventana Win32 propia**;
- aloja un control **WebView2** que ocupa el área cliente;
- renderiza dentro de él la interfaz **React + TypeScript** ya diseñada;

- ejecuta el **núcleo C++20/23 en el mismo proceso**, en sus hilos dedicados;
- **arranca la recepción UDP solo**, sin ninguna acción del usuario.

El navegador y el acceso desde la LAN quedan **fuera de alcance** (OUT-10, OUT-11). Electron queda descartado salvo que aparezca una razón técnica muy fuerte, que hoy no existe.

Cómo se comunican núcleo e interfaz

Se conserva un **socket en bucle local**, con cuatro cambios que lo convierten en infraestructura interna en lugar de en una interfaz pública:

Antes (v1)	Ahora (v2)
Puerto fijo `8787`, configurable	**Puerto efímero** asignado por el sistema, ni fijo ni configurable
Abierto a cualquier cliente local	**Token de un solo uso**, entregado únicamente al WebView2 propio
Opción de exponerlo a la LAN	**No existe esa opción**
Servidor HTTP para los recursos estáticos	**Eliminado.** Los recursos se sirven por mapeo de nombre de host virtual del WebView2 a la carpeta local

Por qué no el paso de mensajes propio del WebView2. Sería lo natural para el canal de control, pero la telemetría de alta frecuencia son tramas **binarias** a 20-60 Hz, y el socket las transporta de forma nativa. Mantenerlo evita reescribir el transporte, conserva el diseño de canales del ADR-006 y deja la puerta abierta a un cliente alternativo sin rediseñar nada. Es una concesión pragmática, y por eso se acompaña de la medición Q-10 en la fase F1.6b: si el WebView2 sostuviera peor el caudal binario que un navegador, conviene saberlo pronto y no al final.

Por qué desaparece el servidor HTTP. Porque el WebView2 puede mapear un nombre de host virtual a una carpeta del disco. Un componente menos, un socket menos escuchando, y la afirmación «esto no es un producto web» pasa a ser literalmente cierta.

Razones

1. **El producto que se quiere es de escritorio.** Ninguno de los nueve puntos de la experiencia objetivo se cumple bien desde un navegador.
2. **WebView2 conserva toda la inversión en React.** Las pantallas, las gráficas sobre canvas y el mapa siguen siendo exactamente el mismo código.
3. **Es la opción más ligera de las que alojan un motor web.** El runtime viene con Windows 11 y se comparte entre aplicaciones; no se empaqueta un navegador por producto.
4. **Un solo proceso y un solo lenguaje anfitrión.** El SDK de WebView2 tiene API de C++, así que el shell se escribe en el mismo lenguaje que el núcleo. Ni Node, ni Rust, ni un puente entre runtimes.
5. **Reduce la superficie de ataque**: desaparece el servidor HTTP y desaparece la posibilidad de exponer la interfaz a la red.

Consecuencias

Positivas. Producto coherente con lo que se quería · un ejecutable, un icono, una ventana · sin pasos manuales · **menos** componentes que en la v1 · misma interfaz React.

Negativas, y asumidas.

- **Dependencia del WebView2 Runtime.** Windows 11 lo trae; Windows 10 puede no tenerlo. Se mitiga comprobándolo al arrancar (RF-632) y en el instalador (RNF-095). Riesgo RSK-13.
- **Sólo Windows.** El núcleo sigue compilando en Linux (RNF-091), pero el shell no. Es una consecuencia buscada: la plataforma objetivo siempre fue Windows.

- **Pruebas de extremo a extremo menos trilladas.** El plan es la depuración remota del propio WebView2; hay que verificarlo pronto (Q-11, RSK-15).
- **Se pierde el observador remoto.** Estaba en el roadmap como CU-19 y se retira sin sustituto. Es la concesión más real de este cambio, y es deliberada.
- **Hay que escribir un shell Win32.** Ventana, bucle de mensajes, PPP, instancia única, estado de ventana y ciclo de vida. Estimado en 8-10 días (fase F1.6b).

Lo que este ADR no cambia

El núcleo, los decoders, el modelo de dominio, el motor de estado, la grabación, el replay y la persistencia son **exactamente los mismos**. El cambio afecta a cómo se aloja y se presenta la interfaz, no a cómo se obtienen ni se procesan los datos.

Que un cambio de enfoque de producto de esta magnitud toque tan poco del núcleo es, precisamente, la señal de que las fronteras del ADR-003 estaban bien puestas. Si el parseo o el estado hubieran vivido en el frontend —la alternativa que ADR-003 descartó—, este cambio habría sido una reescritura.

ADR-003 — Pipeline con hilos dedicados y colas acotadas

Estado: Aceptada

Contexto

Un productor rápido y constante (415 datagramas/s) alimenta a consumidores de velocidades muy dispares: el estado (µs), el disco (ms) y un motor web (decenas de ms). La recepción no puede bloquearse jamás.

Alternativas

Opción	En contra
Monohilo con bucle de eventos	Cualquier operación de disco o cliente lento introduce jitter directamente en la recepción
Actores con paso de mensajes generalizado	Indirección y asignaciones en el camino caliente para un problema de un único emisor
Pipeline con colas acotadas	Más código de infraestructura inicial
Todo el parseo en el frontend	Imposible cumplir la latencia y la persistencia

Decisión

Seis hilos con responsabilidad única, comunicados por colas de un productor y un consumidor con **política de desbordamiento declarada por cola**.

Razones

1. Ajusta a la forma real del problema.
2. La contrapresión **nunca sube**: es una propiedad estructural, no una convención.
3. Es la arquitectura de las adquisiciones de telemetría reales, lo que además satisface el objetivo de aprendizaje.
4. Cada cola documenta su capacidad y qué descarta, de modo que la degradación es previsible y medible.

Consecuencias

Más infraestructura por delante (`BufferPool`, `SpscRing`) y obligación de ThreadSanitizer en integración continua. A cambio, la degradación bajo carga es explícita y contabilizada en lugar de silenciosa.

ADR-004 — Decoder propio

Estado: Aceptada

Contexto

Existen librerías de parseo del protocolo en varios lenguajes. Sólo una tiene soporte verificado del formato 2026, y está en C#.

Alternativas

Opción	Veredicto
Adoptar una librería de C++ existente	**No existe** con soporte 2026
Portar la librería de C#	El esfuerzo de portar es comparable al de transcribir la especificación, con peor trazabilidad
Copiar código con licencia MIT	Compatible legalmente, pero la transcripción directa es más corta y más auditable
Implementación propia desde la especificación	Adoptada

Decisión

Decoder propio, transcrito directamente de la especificación oficial.

Razones

1. **No hay alternativa madura para el stack elegido.** Ésta es la razón determinante; el resto son ventajas añadidas.
2. **Trazabilidad total:** cada struct se corresponde línea a línea con la especificación y el compilador verifica su tamaño.
3. **Control de la seguridad de memoria**, que es el riesgo principal del lenguaje elegido.
4. **Cumple el objetivo formativo** (OBJ-T1).
5. **El coste de mantenimiento es predecible:** los cambios de EA siguen cuatro patrones, todos de transcripción directa (`03-UDP-PROTOCOL.md` §23.1).

Consecuencias

Coste inicial alto: ~33 structs entre ambos formatos. Se compensa con los `static_assert` y los fixtures, que convierten la transcripción en un trabajo verificable en lugar de en un acto de fe.

Se asume la responsabilidad de seguir las revisiones de EA. Mitigado por la detección temprana de `03-UDP-PROTOCOL.md` §23.3.

ADR-005 — Almacenamiento en cuatro capas

Estado: Aceptada

Contexto

Una sesión de 1 h a 60 Hz genera ~1,5 GB de UDP crudo y ~5,2 M de muestras por serie. Los patrones de acceso son radicalmente distintos: escritura secuencial pura para el crudo, transaccional pequeña para el catálogo, por lotes columnar para las series.

Alternativas

Opción	Veredicto
Sólo crudo	Consultar «la vuelta 12» exige reproducir la sesión entera
Sólo normalizado	Un fallo del decoder sería una pérdida de datos irreversible
Una única base de datos para todo	Ninguna es buena a la vez para 5 M de filas de series y para un catálogo transaccional
Servidor de series temporales (InfluxDB, TimescaleDB)	Obliga a instalar y administrar un servidor. Desproporcionado para un usuario en un PC
Cuatro capas especializadas	Adoptada

Decisión

Capa	Tecnología	Contenido
L0	`.f1raw` propio + zstd	Datagramas crudos
L1	Base embebida transaccional	Catálogo
L2	Columnar comprimido particionado	Series de alta frecuencia
L3	Ficheros derivados	Cachés regenerables

Razones

1. Cada capa hace exactamente aquello en lo que es la mejor opción.
2. **Ninguna obliga a instalar un servidor** (P4).
3. Las retenciones pueden ser distintas, que es lo que hace asumible el coste del crudo.
4. El formato columnar es legible por herramientas externas: los datos no quedan encerrados.

Consecuencias

Cuatro tecnologías que mantener y cuatro estrategias de migración. Es más complejo que una única base de datos, y es la complejidad correcta: unificarlas destruiría precisamente lo que las hace adecuadas.

ADR-006 — Transporte interno con canales de garantías distintas

Estado: Aceptada · **revisado por ADR-002 v2**: el transporte deja de ser una interfaz pública y pasa a ser infraestructura interna del proceso (puerto efímero, token de un solo uso, sin exposición a la red)

Contexto

La interfaz necesita a la vez actualizaciones de alta frecuencia y tolerantes a pérdida (posiciones), y eventos discretos e irrepetibles (penalizaciones, banderas).

Alternativas

Opción	En contra
Sondeo HTTP	Latencia incompatible con F3
Eventos enviados por el servidor (SSE)	Unidireccional; sin binario eficiente
Socket en bucle local con canales diferenciados	Adoptada
gRPC-Web	Complejidad de herramientas desproporcionada
Memoria compartida	Ata la interfaz al mismo proceso. Tras ADR-002 v2 eso ya es cierto, pero obligaría a reescribir el transporte y a resolver a mano la sincronización
Paso de mensajes propio del WebView2	Natural para el canal de control, pero la telemetría de alta frecuencia es binaria; el socket la transporta de forma nativa (ADR-002 v2)

Decisión

Un socket en bucle local con seis canales lógicos, cada uno con su garantía: `session`, `timing`, `telemetry`, `events`, `diagnostics`, `replay`. JSON para control y estado; **binario compacto** para telemetría.

Razones

1. **La distinción entre coalescible y no coalescible es la decisión clave.** Una posición antigua no vale nada; una penalización perdida es un defecto.
2. El binario reduce el tráfico de telemetría de ~288 KB/s a ~43 KB/s y elimina el análisis sintáctico por muestra en el navegador.
3. El JSON en los canales de baja frecuencia mantiene la depuración cómoda.
4. Habilita clientes adicionales sin trabajo extra.

Consecuencias

Dos formatos de serialización que mantener. Se acota versionando el esquema binario y generando los decodificadores TypeScript desde el mismo origen que los tipos del dominio.

ADR-007 — Replay como requisito del MVP

Estado: Aceptada

Contexto

Desarrollar contra el juego en vivo exige conducir una carrera por cada iteración. Es inviable y produce pruebas irrepetibles.

Alternativas

Opción	En contra
Desarrollar sólo contra el juego	Inviable en la práctica; pruebas no reproducibles
Datos simulados escritos a mano	Prueban lo que imaginamos, no lo que el juego hace
Replay como añadido posterior	Obligaría a reescribir la fuente de datos, la persistencia y las pruebas
Replay en el MVP	Adoptada

Decisión

Grabación cruda y motor de replay determinista forman parte del MVP, y el replay se implementa **antes** que el motor de estado.

Razones

1. Es la condición que hace viable OBJ-06.
2. `IDatagramSource` con dos implementaciones desde el principio evita una refactorización mayor.
3. El determinismo convierte un fallo intermitente en un fallo reproducible (RSK-07).
4. Es la base de las pruebas de integración reales, no de maquetas.

Consecuencias

Una semana de trabajo antes de tener nada visible. Es la inversión con mejor retorno de todo el proyecto: a partir de F1.4 el juego deja de ser necesario para el desarrollo diario.

ADR-008 — Registro de decoders por `m_packetFormat`

Estado: Aceptada

Contexto

Deben coexistir dos formatos y debe poder añadirse un tercero sin propagar condicionales.

Alternativas

Opción	En contra
`if (format == 2026)` en los puntos de uso	Se multiplica por todo el código; es exactamente lo que hay que evitar
Plantillas parametrizadas por formato	Elegante, pero el formato se conoce en ejecución, no en compilación
Registro de implementaciones de una interfaz	Adoptada

Decisión

`IProtocolDecoder` con una implementación por formato, registradas en un `DecoderRegistry` consultado por `m_packetFormat`.

Razones

1. Añadir un formato requiere un decoder, sus catálogos y su registro. **Nada más** (RNF-040).
2. El coste en ejecución es una llamada virtual por datagrama: irrelevante frente a los 10-30 µs de decodificación.
3. Las diferencias semánticas (`ersDeployMode` 3) se resuelven dentro del decoder, no en el dominio.

Consecuencias

Una llamada virtual y algo de duplicación entre decoders. La duplicación es **deseable**: dos formatos que hoy se parecen pueden divergir mañana, y unificarlos prematuramente crearía un acoplamiento que habría que deshacer.

ADR-009 — `Channel<T>`: procedencia obligatoria en el tipo

Estado: Aceptada

Contexto

El error más frecuente del ecosistema es tratar un cero de telemetría restringida como un cero real, produciendo interfaces que muestran «0 % de desgaste» de forma falsa.

Alternativas

Opción	En contra
Convención y disciplina	Falla en cuanto hay prisa. Ha fallado en todas las herramientas del ecosistema
`std::optional<T>` solo	Distingue presente de ausente, pero no **por qué** está ausente
Un campo de procedencia paralelo	Se puede olvidar; nada obliga a mantenerlos sincronizados
`Channel<T>` que encapsula valor y procedencia	Adoptada

Decisión

Todo canal de dominio es un `Channel<T>` sin conversión implícita a `T`. Obtener el valor exige comprobar `known()`.

Razones

1. Convierte una convención en una **restricción del compilador**.
2. Distingue las cinco procedencias, que informan de cosas distintas al usuario.
3. Se propaga naturalmente hasta el transporte y la interfaz: el JSON lleva la procedencia y el binario lleva la máscara de presencia.

Consecuencias

Código algo más verboso en el dominio. A cambio, RN-001 pasa de ser un requisito que hay que recordar a ser un invariante que no se puede violar. Es un intercambio claramente favorable.

ADR-010 — El grabador va antes del decoder

Estado: Aceptada

Contexto

El esquema inicial situaba el grabador después del motor de estado.

Decisión

`RawRecorder` se conecta a la salida del receptor, **antes** de la cabecera y del decoder.

Razones

1. **Se graba lo que llegó, no lo que entendimos.** Un fallo del decoder no contamina la captura.
2. Permite el reproceso con decoders futuros (RF-413).
3. Permite grabar **también los datagramas inválidos**, que son la evidencia de un cambio de protocolo.
4. Simplifica el grabador: no depende de nada más que de bytes y una marca de tiempo.

Consecuencias

La captura ocupa más que un formato normalizado (~430 MB/h frente a ~70 MB/h). Se compensa con retenciones distintas por capa (ADR-005): el crudo se archiva o se descarta, el normalizado se conserva.

ADR-011 — Formato de captura propio en lugar de pcap

Estado: Aceptada

Contexto

pcap es el formato estándar de captura de red y tiene herramientas maduras.

Alternativas

Opción	En contra
pcap / pcapng	Guarda cabeceras de red irrelevantes; sin puntos de sincronización para búsqueda temporal; sin metadatos del dominio; sin compresión por bloques integrada
Formato propio	Hay que escribir escritor, lector, índice y migraciones

Decisión

Formato `.f1raw` propio, con especificación publicada y herramienta de línea de órdenes.

Razones

1. Los puntos de sincronización y las instantáneas de estado son **necesarios** para RF-408 y no existen en pcap.
2. La cabecera con formato de protocolo, versión del juego y configuración es información de dominio que pcap no puede llevar.
3. La compresión por bloques con búsqueda es integral al diseño.
4. Publicar la especificación y una herramienta compensa la pérdida de interoperabilidad.

Consecuencias

Nadie puede abrir un `.f1raw` con herramientas de red genéricas. Se mitiga con `pw-capture` y con la exportación a formatos estándar (F2).

ADR-012 — Sin soporte del formato 2024

Estado: Aceptada

Contexto

El juego ofrece los formatos 2026, 2025 y 2024.

Decisión

Sólo se soportan 2025 y 2026.

Razones

1. **Sin caso de uso.** El usuario objetivo juega el 2026 Season Pack; el 2025 se soporta por el juego base y por compatibilidad de capturas.
2. Cada formato añadido multiplica structs, catálogos, fixtures y pruebas.
3. El registro de decoders permite añadirlo después si aparece la necesidad, sin rediseño.

Consecuencias

Un usuario con el formato 2024 configurado verá un aviso claro y una instrucción para cambiarlo. Es una limitación consciente y comunicada, no un fallo.

ADR-013 — Gráficas sobre canvas, no SVG

Estado: Aceptada

Contexto

Una vuelta a 60 Hz tiene ~5000 muestras; una comparación con ocho canales, ~80 000 puntos.

Decisión

Librería de gráficas sobre canvas, con decimado que **conserva mínimo y máximo por columna de píxel**.

Razones

1. SVG crearía decenas de miles de nodos del DOM: inviable a 60 fps.
2. El decimado por muestreo simple **haría desaparecer los picos de frenada**, que es exactamente lo que se quiere ver. Conservar mínimo y máximo lo evita.
3. Requisito no negociable: series con huecos que **no** se interpolan.

Consecuencias

Se pierde la inspección de elementos del motor web sobre los puntos, y las interacciones (cursor, zoom) hay que implementarlas explícitamente. Aceptable frente al beneficio.

ADR-014 — Mapa reconstruido desde telemetría

Estado: Aceptada

Contexto

Las herramientas del sector suelen usar mapas dibujados a mano.

Alternativas

Opción	En contra
Imágenes dibujadas a mano	Trabajo por cada circuito; los inversos son otro trabajo; un circuito nuevo (Madrid) requiere una imagen nueva; riesgo de marca si se toman de fuentes ajenas
Reconstrucción desde `worldPosition`	Se pierde la elevación; requiere una vuelta válida completa

Decisión

Reconstrucción desde la telemetría, cacheada por `(trackId, protocolFormat)`.

Razones

1. **Cualquier circuito funciona automáticamente**, incluidos los nuevos y los inversos.
2. Evita incorporar activos gráficos de terceros (`01-REQUIREMENTS.md` §26.3).
3. Habilita comparar trazadas sobre el mismo sistema de coordenadas.
4. Se corresponde con la línea **realmente recorrida**, que es más informativa que un contorno idealizado.

Consecuencias

La primera vuelta en un circuito nuevo muestra una nube de puntos progresiva en lugar de un mapa terminado. Se comunica con un estado explícito de «reconstruyendo trazado». La pérdida de elevación se documenta.

ADR-015 — Fuente de verdad: el PDF de mayor versión

Estado: Aceptada

Contexto

EA publica varias fuentes oficiales del mismo formato, y **no coinciden**: el `.txt` de structs 2026 omite `PartialModeEnabled` y cuatro códigos de evento que el PDF v1.2 sí documenta.

Alternativas

Opción	En contra
Preferir el `.txt` por ser código	Está incompleto y desactualizado
Implementar la unión de ambas	No resuelve un conflicto real, sólo una omisión
Preferir el PDF de mayor versión	Adoptada
Preferir el comportamiento observado	Sólo aplicable cuando existe una captura del caso

Decisión

Precedencia: **PDF de mayor versión** > `.txt` de structs > comportamiento observado en capturas reales, salvo que la captura contradiga al PDF, en cuyo caso se documenta la discrepancia y se implementa lo observado.

Razones

1. El PDF v1.2 lleva número de versión y nota de revisión; el `.txt` no.
2. El **changelog oficial corrobora** los cuatro códigos por una vía independiente.
3. El comportamiento real del juego es la autoridad última, pero sólo cuando hay evidencia capturada.

Consecuencias

Se implementan 25 códigos de evento en 2026 en lugar de los 21 del `.txt`. Si alguno resulta no emitirse nunca, no habrá coste: el decoder los reconoce y simplemente no llegarán.

ADR-016 — Docker: no para ejecutar ni distribuir; opcional en integración continua

Estado: Aceptada · 2026-09-11

Contexto

Docker no se había mencionado en ninguno de los 18 documentos. No era una decisión tomada: era una omisión. Conviene cerrarla explícitamente, porque en muchos proyectos se asume por defecto y aquí sería un error.

PitWall es una aplicación de escritorio para Windows que recibe UDP de un juego que corre **en la misma máquina**, escribe en el disco local y debe abrirse con un doble clic.

Alternativas

Opción	A favor	En contra
No usar Docker para el producto	El usuario instala una cosa y funciona	Ninguna: no aporta nada al caso de uso
Distribuir PitWall como contenedor	Entorno reproducible	El juego emite a `127.0.0.1`. Docker Desktop en Windows corre sobre WSL2 con red traducida por NAT: recibir UDP del host —y más aún en modo broadcast, que es el escenario de consola— va de incómodo a inviable. Además exige instalar Docker Desktop, contradice P4 y RNF-098, y complica el acceso al disco local y a la ventana
Contenedor sólo para el núcleo, shell fuera	«Aísla» el backend	Parte en dos lo que es un único proceso (ADR-002) y hereda todos los problemas de red del punto anterior sin ninguna ventaja
Docker sólo en integración continua	Compilación y pruebas en Linux reproducibles	Una dependencia más en el entorno de desarrollo, no en el producto

Decisión

1. **Docker NO se usa para ejecutar PitWall.** Ni en desarrollo, ni en pruebas manuales, ni en producción.
2. **Docker NO se usa para distribuir PitWall.** La distribución es un instalador de Windows más una variante portable (RNF-093).
3. **Docker NO es un requisito para el usuario.** Registrado como OUT-12 y RNF-098.
4. **Docker PUEDE usarse, opcionalmente, en integración continua**, para el trabajo que en Windows es incómodo: compilar con Clang en Linux (RNF-091) y ejecutar ThreadSanitizer, que sobre Windows tiene un soporte limitado.
5. Ese uso es **opcional**: la suite completa debe poder ejecutarse en Windows sin Docker, y así se verifica.

Razones

1. **La red lo hace inviable.** Un contenedor no está en la misma pila de red que el juego. Todo el diseño del receptor —`127.0.0.1`, broadcast, `SO_RCVBUF` de 4 MB, medición de pérdida— asume acceso directo al socket del host.
2. **Contradice la experiencia objetivo.** El punto 10 dice literalmente que el usuario no debe necesitar Docker.
3. **No resuelve ningún problema real.** Las dependencias de C++ ya son reproducibles por manifiesto con versiones fijadas (RNF-092); el aislamiento que aporta un contenedor no hace falta en una aplicación local de un solo usuario.
4. **Sí resuelve uno concreto en integración continua.** ThreadSanitizer es un requisito duro (RNF-054, SEC-17) y su camino cómodo es Linux con Clang. Ahí un contenedor es la herramienta adecuada.

Consecuencias

Positivas. Instalación trivial para el usuario · sin problemas de red ni de acceso a disco · una decisión menos que reconsiderar en cada fase.

Negativas, y asumidas.

- El entorno de desarrollo en Windows depende de lo que haya instalado localmente. Se mitiga con el manifiesto de dependencias con versiones fijadas y con la verificación de instalación en máquina limpia (A-36).
- La integración continua tiene dos caminos —Windows nativo y contenedor Linux— y hay que mantener ambos. Se acepta porque el segundo es el único que da ThreadSanitizer con comodidad.

Nota sobre el vocabulario

En esta documentación la palabra «contenedor» aparece también con otros sentidos: el *contenedor de inyección de dependencias* (descartado en ADR-003) y el *contenedor del formato* `.f1raw`. Ninguno tiene relación con este ADR.

Plantilla para ADR futuros

```
## ADR-0NN — Título

**Estado:** Propuesta | Aceptada | Sustituida por ADR-0MM | Obsoleta
**Fecha:** AAAA-MM-DD

### Contexto
Qué fuerzas actúan y por qué hay que decidir ahora.

### Alternativas
Tabla con opciones, a favor y en contra. Al menos tres, incluida «no hacer nada».

### Decisión
Una frase inequívoca.

### Razones
Numeradas, de mayor a menor peso.

### Consecuencias
Positivas y negativas. **Las negativas son obligatorias**: un ADR sin consecuencias
negativas es un ADR que no ha pensado lo suficiente.
```

Fin de `PW-ADR-015` *v1.1*